Advanced UCLCHEM modeling

Facilitating easier modeling

Object Oriented models

- New python system, creates an object of a model
- Model objects have access to built in methods
- Why do this?
  - Simplifies maintenance
  - Simplifies adding new model types
- Why you should care?
  - Simpler to use
  - Simplifies integrating UCLCHEM into other codes

Coding tangent

- A Class, is a defined ‘blueprint’ to create an object.
- Classes define attributes and methods an object will have.
  - Attribute: Variable held by an object.
  - Method: Function specifically defined for a class within which attributes of the class can be accessed.
- Classes can inherit from other classes.
  - Leverage power of a general class, to define specific class.
- Objects can be passed to each other.

What does this mean for UCLCHEM?

- AbstractModel class, defines attributes, and methods all models need
- All models, inherit from AbstractModel.
  - AbstractModel cannot be run, it serves only as a blueprint for other classes
- Object to Object interactions
  - Starting chemistry for a new model from a previous model is simplified to passing object
- Methods are built in and are universal.
  - Methods added to abstract class, are usable by any model type object

Legacy v. Object: Cloud

Running a single model:

- Similarities
  - Same param_dict
  - Similar way to run
- Differences
  - Accessing all attributes post run
  - cloud.get_dataframes() returns df_stage1_physics equivalent

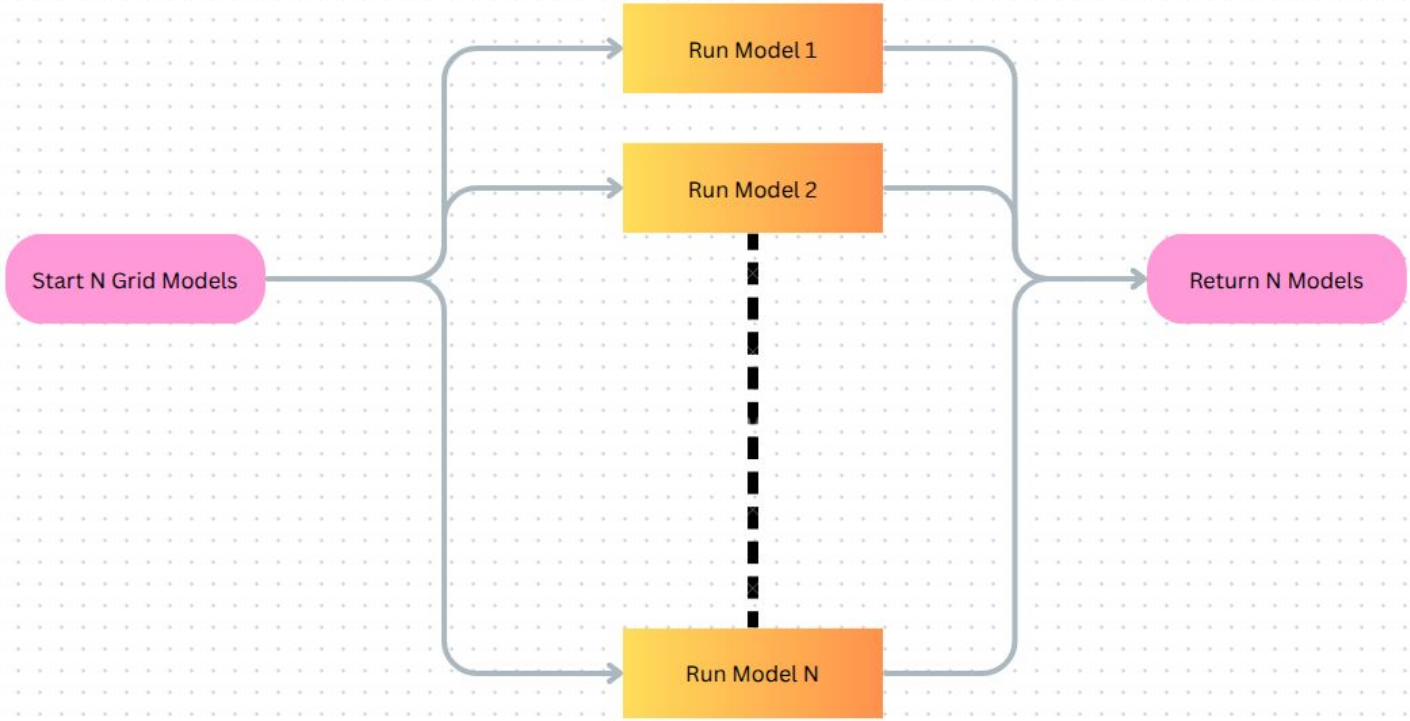
```
# set a parameter dictionary for cloud collapse model
param_dict = {
    "endAtFinalDensity": False,  # stop at finalTime
    "freefall": True,  # increase density in freefall
    "initialDens": 1e2,  # starting density
    "finalDens": 1e6,  # final density
    "initialTemp": 10.0,  # temperature of gas
    "finalTime": 6.0e6,  # final time
    "rout": 0.1,  # radius of cloud in pc
    "baseAv": 1.0,  # visual extinction at cloud edge.
}
df_stage1_physics, df_stage1_chemistry, df_stage1_rates, final_abundances, result = (
    uclchem.model.cloud(
        param_dict=param_dict,
        return_dataframe=True,
    )
```

```
param_dict = {
    "endAtFinalDensity": False,  # stop at finalTime
    "freefall": True,  # increase density in freefall
    "initialDens": 1e2,  # starting density
    "finalDens": 1e6,  # final density
    "initialTemp": 10.0,  # temperature of gas
    "finalTime": 6.0e6,  # final time
    "rout": 0.1,  # radius of cloud in pc
    "baseAv": 1.0,  # visual extinction at cloud edge.
}
cloud = uclchem.model.Cloud(param_dict=param_dict)
```

Advanced Runners

- SequentialRunner
  - Various model types run in series
  - Results of first model used to initiate second, second for third…
- GridRunner
  - Multiple models of same type run in parallel
  - Models do not chemically interact with another
  - Save models to file as calculated

Start N Serial Models
Run Model 1
Run Model 2
Run Model N
Return N Models


Start N Grid Models
Run Model 1
Run Model 2
Run Model N
Return N Models

Benefits of Advanced Runners

- SequentialRunner
  - Chemical abundances for the next model is automated
  - Passing of modeled final density and temperature are an option
- GridRunner
  - Option to generate parameter grid based on given lists/arrays
  - Multiprocessing running of models
    - Running 10 models with 10 workers takes as long as running the longest model in the 10.
  - Option to force given grid
    - I.e. “I have these N particles in my simulation with x, y, z parameters that I can give to uclchem.model.GridRunner”tt

Questions?

For the exercises

In your UCLCHEM git directory

- git fetch origin
- git checkout workshop_hot_patches_day4
- git pull
  - just in case, should say ‘Already up-to-date.`
- pip install .